

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Corso di Laurea Magistrale in Informatica

Software Engineering for AI 2025/2026

Prof.:
Palomba Fabio

Credit:
**Federico Pomposelli
Gaetano Vito Faraco**

ANNO ACCADEMICO 2025/2026

Indice

Sommario

Nel moderno panorama dell'ingegneria del software orientata all'Intelligenza Artificiale (SE4AI), gli strumenti di programmazione agentica hanno trasformato i modelli linguistici da semplici motori di completamento a contributori autonomi capaci di navigare repository e risolvere problemi complessi. Al centro di questo paradigma si collocano i *Context File* (come `AGENTS.md` o `CLAUDE.md`), artefatti software che fungono da manuali di istruzioni permanenti a livello di progetto. Questo studio analizza come tali file agiscano sul comportamento degli agenti, la loro aderenza alle specifiche e l'impatto sull'efficienza operativa.

DEFINIZIONE DI OBIETTIVI/GOALS

L'obiettivo primario è caratterizzare i *Context File* come componenti dinamici del ciclo di vita del software. Nello specifico, la ricerca mira a:

- **Caratterizzazione Strutturale e Manutentiva:** Analizzare la dimensione, la leggibilità e la frequenza di aggiornamento di tali file, verificando se evolvono coerentemente con il codice sorgente.
- **Analisi dell'Aderenza Semantica:** Valutare come lo stile di scrittura (descrittivo, prescrittivo, proibitivo) influenzi l'aderenza dell'agente ai vincoli di progetto.
- **Valutazione dell'Efficienza:** Misurare l'impatto dei file sulla latenza e sul consumo di risorse computazionali durante task di sviluppo reali.
- **Identificazione dei Punti Ciechi:** Investigare la scarsità di Requisiti Non Funzionali (NFR), quali sicurezza e prestazioni, all'interno dei manifesti attuali.

Definiti gli obiettivi del sistema, ne ricaviamo in primo luogo dei requisiti non-funzionali: la manutenibilità, l'efficienza, Sicurezza/Robustezza, affidabilità e performance. Su di essi orienteremo il design del sistema. Assumiamo che gli ACF (Agent Context Files) e la loro stesura rappresentino un passaggio fondamentale per il soddisfacimento dei requisiti.

DATA COLLECTION

Il processo di data collection si basa sull'estrazione di informazioni da repository open-source (tramite il dataset AIDev) per garantire condizioni sperimentali realistiche.

- **Selezione del Campione:** Si considerano repository che adottano un unico file `AGENTS.md` nella radice per minimizzare variabili di confusione derivanti da istruzioni sovrapposte.
- **Criteri di Inclusione PR:** Verranno selezionate Pull Request (PR) mergeate che modificano esclusivamente il codice, con un limite di 100 righe di codice (LoC) e non più di 5 file coinvolti per garantire la ripetibilità.
- **Ricostruzione Ambientale:** Per ogni task, il repository verrà riportato allo stato pre-merge tramite *checkout* del commit specifico, isolando la versione di `AGENTS.md` corrispondente.
- **Standardizzazione dei Task:** Le descrizioni dei compiti verranno standardizzate in stile "GitHub Issue" utilizzando un LLM locale, fornendo all'agente un input informativo e coerente.

PROGETTAZIONE ARCHITETTURA/DESIGN DEL SISTEMA

Forniamo una preliminare architettura dell'agente, che riflettano i goal stabiliti:

- **Context-Aware Orchestrator:** Gestisce gli ACF come memoria a lungo termine, ottimizzando il caricamento dei vincoli per massimizzare l'aderenza semantica.

- **Reasoning Engine (LLM):** Utilizzo di modelli (Claude/GPT) tramite API, o OpenSource (Mistral), valutati per la capacità di seguire istruzioni sistemiche.
- **Tool Layer (AIDev):** Tooling per l'interazione con il file system e il terminale.
- **Governance Layer (Architectural Guardrails):** Implementazione di filtri che validano l'output rispetto agli NFR (Sicurezza/Performance) definiti nell'ACF prima dell'esecuzione.

TESTING e VALUTAZIONE

L'analisi dei dati seguirà un approccio quantitativo e qualitativo strutturato.

- **Metriche Operative:** Verranno confrontate le esecuzioni "con" e "senza" context file misurando il tempo di completamento (*wall-clock time*) e l'uso totale dei token (input, cache e output).
- **Tassonomia delle Istruzioni:** Il contenuto verrà classificato in 16 categorie distinte (es. *Architecture, Build and Run, Test*) per identificare quali aree tematiche ricevano maggiore enfasi dagli sviluppatori.
- **Analisi della Leggibilità e Debito Tecnico:** Si utilizzerà la metrica *Flesch Reading Ease* (FRE) per misurare la complessità linguistica. Questo permetterà di valutare il "Debito di Contesto" (*Context Debt*), ovvero il paradosso per cui istruzioni eccessivamente complesse aumentano il carico cognitivo umano pur essendo destinate alle macchine.
- **Validazione dell'Aderenza:** Si analizzerà se la presenza di categorie funzionali concrete (es. *Build and Run*) riduca la navigazione esplorativa dell'agente, portando a una generazione di codice più efficiente e mirato.
- **Test Metamorfici:** Verifica della robustezza variando lo stile linguistico dell'ACF a parità di obiettivo per misurare la divergenza nelle decisioni dell'agente.
- **Testing correttezza funzionale:** l'esecuzione di unit test automatizzati per accertare la correttezza funzionale.
- **Validation tramite protocollo di 'LLM-as-a-Judge':** per misurare l'aderenza semantica del codice prodotto ai vincoli non-funzionali definiti nell'ACF. Si intende usare un modello superiore rispetto al modello dell'agente per migliorarne la validità.

DEPLOYMENT E MONITORAGGIO Per garantire i criteri di osservabilità e affidabilità in produzione, abbiamo individuato una serie di strumenti che possano soddisfare questa esigenza:

- **Osservabilità (Tracing):** Utilizzo di framework come *LangSmith* o *Arize Phoenix* per il tracing granulare dei passaggi di ragionamento dell'agente. Questo permette di isolare se un fallimento derivi da un'istruzione ambigua nell'ACF o da un limite del modello (LLM).
- **Monitoring dei Guardrails:** Implementazione di un layer di monitoraggio real-time (es. *NeMo Guardrails*) che intercetta violazioni dei vincoli di sicurezza e performance definiti negli ACF, generando alert in caso di tentativi di esecuzione di codice non sicuro.
- **Infrastructure-as-Code (IaC):** Deployment containerizzato, dove la configurazione dell'ambiente di esecuzione (AIDev) e gli ACF sono versionati congiuntamente (Asset Level 2), garantendo la piena riproducibilità operativa.
- **Feedback Loop:** Valutiamo tenere in considerazione il fenomeno "human-in-the-loop" e di come l'essere umano influenzi il lavoro dell'agente.

RAGGIUNGIMENTO dei GOALS

Lo studio intende dimostrare che il *Context Engineering* non è un'attività di documentazione statica, ma una nuova forma di "Configuration-as-Code". Ci si aspetta di confermare che un

Context File ben strutturato riduca il consumo di token (stimato in circa il 20%) e i tempi di esecuzione, evidenziando però la necessità critica di integrare sezioni dedicate alla sicurezza per evitare la generazione di codice vulnerabile.